

Về việc xin thực hiện Đồ án 1 học kỳ 2 năm học 2025-2026

15 messages

Nguyễn Minh, Hồ <23520923@gm.uit.edu.vn>
To: yentth@uit.edu.vn

Sun, Dec 28, 2025 at 9:42 AM

Em kính chào cô,

Em là Hồ Nguyễn Minh (23520923),
em hiện đang học lớp Công nghệ Web và ứng dụng của cô (SE347.Q14).

Em gửi thư này xin phép cô được thực hiện Đồ án 1 kỳ tới (kỳ 2 2025-2026) với cô.

Em có dự tính sẽ thực hiện một dự án stream nhạc trực tuyến (Spotify, Youtube Music), backend .NET và frontend có thể web hoặc native mobile. Em dự tính deploy app lên K8s, với IaC (Terraform), và nếu đủ thời gian sẽ cố gắng phủ test coverage cho unit test.

Tuy nhiên, ý tưởng trên chỉ là dự tính ban đầu, nếu cô có góp ý em có thể thay đổi đồ án ạ.

Em biết hiện đã có nhiều bạn khác mail cô xin đăng ký đồ án từ trước, nhưng dù nếu cô không nhận em cũng mong được nhận ý kiến của cô về đồ án.

Em xin cảm ơn cô,
Hồ Nguyễn Minh

Trần Thị Hồng, Yến <yentth@uit.edu.vn>
To: "Nguyễn Minh, Hồ" <23520923@gm.uit.edu.vn>

Wed, Dec 31, 2025 at 12:16 PM

Chào Minh,

Dự án thể hiện tư duy kỹ thuật hiện đại, bám sát các tiêu chuẩn công nghiệp khi kết hợp sức mạnh của **.NET** với hệ sinh thái **Cloud Native**. Việc sử dụng **Kubernetes (K8s)** và **Terraform (IaC)** không chỉ giúp hệ thống vận hành ổn định, dễ mở rộng mà còn khẳng định năng lực triển khai hạ tầng chuyên nghiệp của sinh viên.

Định hướng phát triển:

- **Hạ tầng:** Hoàn thiện quy trình **CI/CD** tự động từ khâu đẩy code đến triển khai lên K8s nhằm tối ưu hóa vòng đời sản phẩm.
- **Tính năng:** Nghiên cứu giao thức phát trực tuyến như **HLS** để tối ưu tốc độ truyền tải nhạc và trải nghiệm người dùng.
- **Chất lượng:** Ưu tiên phủ **Unit Test** cho các logic quan trọng và tích hợp các công cụ giám sát (**Prometheus/Grafana**) để theo dõi sức khỏe hệ thống trên K8s.

Đây là nền tảng vững chắc để phát triển thành một hệ thống microservices hoàn chỉnh trong tương lai khi phát triển lên Khóa luận tốt nghiệp.

Lộ trình xây dựng và phát triển gợi ý như sau:

1. Đồ án 1: Xây dựng nền tảng (Foundation)

Mục tiêu là hoàn thiện luồng nghiệp vụ cơ bản và làm quen với hạ tầng Cloud Native.

- **Backend & DB:** Thiết kế Database (SQL Server/PostgreSQL), xây dựng API CRUD cho bài hát, nghệ sĩ và danh sách phát bằng **.NET** (Clean Architecture).
- **Streaming:** Triển khai stream nhạc đơn giản bằng cách trả về luồng dữ liệu (stream) từ **Object Storage (MinIO)**.

- **DevOps:** Đóng gói ứng dụng bằng **Docker**. Sử dụng **Terraform** để dựng cụm K8s (local hoặc cloud) và deploy thủ công.
- **Frontend:** Web cơ bản (React) để phát nhạc và quản lý thư viện cá nhân.

2. Đề án 2: Tối ưu hóa và Hệ thống hóa (Optimization)

Tập trung vào hiệu suất, trải nghiệm người dùng và tự động hóa vận hành.

- **Streaming nâng cao:** Chuyển đổi sang giao thức **HLS (HTTP Live Streaming)** để nhạc phát mượt mà, hỗ trợ thay đổi chất lượng theo tốc độ mạng.
- **Caching & Search:** Tích hợp **Redis** để cache các bài hát phổ biến và **Elasticsearch** để tìm kiếm nhanh theo tên, lời bài hát.
- **CI/CD:** Xây dựng luồng tự động (GitHub Actions/GitLab CI) để tự động chạy **Unit Test** và deploy lên K8s mỗi khi có thay đổi code.
- **Frontend:** Nâng cấp giao diện chuyên nghiệp hơn hoặc phát triển bản **Native Mobile** (Flutter/React Native).

3. Khóa luận tốt nghiệp: Quy mô và Trí tuệ nhân tạo (Scalability & AI)

Đây là giai đoạn đưa dự án lên tầm một sản phẩm thực tế với độ phức tạp cao.

- **Microservices:** Tách các dịch vụ (Auth, Streaming, Payment, Notification) thành các microservices độc lập để dễ dàng mở rộng.
- **Hệ thống gợi ý (Recommendation System):** Áp dụng Machine Learning để gợi ý nhạc theo sở thích người dùng (User-based/Content-based Filtering).
- **Observability:** Cài đặt **Prometheus & Grafana** để giám sát hiệu năng hệ thống trên K8s; tích hợp ELK Stack để quản lý log.
- **Security:** Triển khai **API Gateway** (Ocelot hoặc Kong), bảo mật bằng OAuth2/OpenID Connect và quản lý Secrets qua Vault.

Em tham khảo lộ trình gợi ý trên hoặc đề xuất lộ trình khác phù hợp thời gian, nhân lực và năng lực để Cô góp ý thêm nhé!

Cô Yến.

[Quoted text hidden]

Nguyễn Minh, Hồ <23520923@gm.uit.edu.vn>

Thu, Jan 1, 2026 at 5:54 PM

To: "Trần Thị Hồng, Yến" <yentth@uit.edu.vn>

Em chào cô.

Em chưa có ý định làm khóa luận tốt nghiệp nên tạm thời kế hoạch của em không bao gồm phần đó mà chỉ xoay quanh DA1 và DA2.

Em dự tính DA1 sẽ làm hệ thống backend monolith (cổ tình không microservice), tuy nhiên các service dễ bottleneck như stream nhạc, statistics cơ bản,... sẽ được tách ra để dễ scale trên K8s. Backend của em sẽ stream nhạc qua các protocol chuyên stream từ ban đầu (có thể là HLS) chứ không chỉ serve file tĩnh qua object storage gửi HTTP raw, do em có dự tính mở rộng stream cả MV đi kèm bài hát. Em sẽ làm CI/CD từ đầu với GitOps, và các best practice như quản secrets bằng vault,... nếu kịp em sẽ thêm observability qua opentelemetry. .NET Aspire trông khá mới, nên em sẽ xem qua thử, nhưng em chưa rõ sẽ dùng không do công nghệ khá mới.

Hiện tại em chưa tìm được ứng dụng khác cho Redis ngoài việc invalidate token, em không muốn cache file trên đó do sẽ có dự định dùng CDN.

Với DA2 em dự tính refactor backend thành microservice để theo đúng hướng phát triển thông thường của các dự án thực tế, chuyển sang microservice khi nghiệp vụ phức tạp hóa. Sau đó, em sẽ tìm cách stream nhạc từ CDN (có quản quyền qua token để tránh lỗi bảo mật), và thêm hệ thống statistics cụ thể bằng các tool như Kafka/Spark,...

Nếu kịp em cũng muốn làm hệ thống lyrics có hỗ trợ hiệu ứng karaoke (hoặc có thể dùng các lib mở như LibLRC). Có thể em sẽ tham khảo implementation của extension BetterLyrics cho Youtube Music.

Phía trên là dự tinh của em ạ. Em cảm ơn cô.
[Quoted text hidden]

Trần Thị Hồng, Yên <yentth@uit.edu.vn>
To: "Nguyễn Minh, Hồ" <23520923@gm.uit.edu.vn>

Sat, Jan 3, 2026 at 12:04 AM

Chào Minh,

1. Về chiến lược kiến trúc (ĐA1 -> ĐA2)

- **Hướng đi Monolith -> Microservices:** là quyết định đúng.
- **Kế hoạch không làm Khóa luận tốt nghiệp (KLTN):** tùy em quyết định. Tuy nhiên, khối lượng công việc em hoạch định cho ĐA2 (Microservices + Kafka/Spark + CDN security) thực chất tương đương KLTN. Nếu em làm tốt ĐA2 thì sản phẩm đó đủ chất lượng để bảo vệ KLTN. Em nên cân nhắc lại thêm về việc thực hiện KLTN.

2. Nhận xét ĐA1:

a. Về Streaming Protocol (HLS):

- **Điểm cộng:** Việc chọn HLS (HTTP Live Streaming) thay vì gửi file raw là điểm cộng lớn nhất. Nó cho phép Adaptive Bitrate Streaming (tự điều chỉnh chất lượng mạng).
- **Lưu ý kỹ thuật:** Để làm được HLS, backend .NET của em sẽ phải tích hợp FFmpeg để transcode file upload (mp3/wav/mp4) thành các file .m3u8 và .ts. Quá trình này rất tốn CPU => Em hãy xử lý bất đồng bộ (Background Job). Khi user upload xong, đẩy job vào hàng đợi (Queue), worker xử lý xong mới báo "Ready". Đừng bắt user chờ transcode xong mới được lưu.

b. Về DevOps (K8s, GitOps, Vault):

- **Cảnh báo:** Em đang ôm đồm quá nhiều thứ về Infra cho ĐA1. Setup K8s + Vault + GitOps (ArgoCD?) chạy mượt mà tốn rất nhiều thời gian => Em hãy chắc chắn Application (Logic code) chạy tốt trên Docker Compose trước. K8s chỉ là nơi deploy. Đừng để việc cấu hình K8s chiếm 80% thời gian làm đồ án trong khi tính năng nghe nhạc lại sơ sài.
- **.NET Aspire:** em nên thử để giải quyết bài toán "Orchestration" cho .NET cloud-native apps. Nó sẽ giúp em visualize được logs, traces, metrics giữa các service (Backend, Redis, Postgres) ngay trên local mà không cần setup Prometheus/Grafana phức tạp từ đầu.

c. Về Redis:

- Em nói: *"chưa tìm được ứng dụng khác cho Redis ngoài việc invalidate token"*. Đây là một sự lãng phí tài nguyên lớn.
- Redis không chỉ để cache file (binary), mà để cache Metadata.
 - Khi user mở app, họ load danh sách "Top 100 bài hát", "Thông tin Artist", "Album detail". Những dữ liệu này là text/json, truy xuất từ SQL Server/PostgreSQL rất tốn kém nếu có hàng nghìn request.
 - Hãy cache các query database nặng vào Redis (VD: JSON của Home Screen). Với chiến lược Cache-aside sẽ giúp backend của em "nhẹ gánh" và scale được.
 - Ngoài ra, Redis cũng dùng cho Rate Limiting (chống spam API), Pub/Sub (cho tính năng realtime lyrics sau này).

3. Nhận xét ĐA2:

a. Big Data (Kafka/Spark):

- **Kafka:** Hợp lý để làm Message Broker xử lý lượt nghe (logs) với lưu lượng lớn.
- **Spark:** Nếu dữ liệu của em chưa đến mức hàng triệu record/ngày thì Spark là quá nặng nề (cần cluster riêng, RAM lớn) => Em có thể dùng Kafka Streams hoặc đơn giản là Time-series Database (như InfluxDB hoặc TimescaleDB) để làm hệ thống Statistics/Analytics. Nó nhẹ hơn và phù hợp với quy mô Đồ án 2 hơn Spark.

b. CDN & Security:

- Từ khóa em cần tìm hiểu là "Signed URLs" (hoặc Signed Cookies) của CloudFront (AWS) hoặc Cloudflare. Backend sẽ ký một cái link có hạn sử dụng (ví dụ 1 tiếng) để frontend play.

c. Lyrics & Karaoke:

- Dùng file .Irc là chuẩn. Logic sync thời gian giữa client (Frontend) và nhạc đang chạy là một thử thách về xử lý bất đồng bộ trên Frontend (JS/Native), không liên quan nhiều đến Backend.

4. Đánh giá mức độ khả thi:

- **Đánh giá:** tốt về mặt định hướng.
- **Rủi ro:** Em tập trung quá nhiều vào hạ tầng (K8s, Vault, Terraform) và kiến trúc (Microservices) mà có thể quên mất trải nghiệm người dùng (UX) và tính năng nghiệp vụ (Playlist, Search, Recommendation).
- **Lời khuyên:**
 1. ĐA1: Tập trung làm mượt tính năng HLS Streaming và Metadata Caching (Redis). Infra chỉ cần Docker Compose hoặc K8s cơ bản, khoan hãy làm Vault/GitOps nếu thấy không đủ thời gian.
 2. Tận dụng Redis để cache dữ liệu JSON (Metadata), đừng chỉ dùng để revoke token.
 3. Thử nghiệm .NET Aspire để đơn giản hóa việc setup môi trường dev.
 4. Bổ sung 1 số tính năng nghiệp vụ để lấy điểm cộng (Playlist, Search, Recommendation)

Em suy nghĩ cân nhắc và có thể ghép nhóm với 1 thành viên khác (nếu muốn) để cùng thực hiện đề tài nhé.
Cô Yến.

[Quoted text hidden]

Nguyễn Minh, Hồ <23520923@gm.uit.edu.vn>
To: "Trần Thị Hồng, Yến" <yentth@uit.edu.vn>

Sat, Jan 3, 2026 at 4:59 PM

Em chào cô.

Trước hết, em xin cảm ơn cô đã cho em thêm nhận xét về đồ án và hướng đi.

Về KLTN, do em học trễ DA1 (kỳ 2 năm 3 mới học), nên nếu làm KLTN em sẽ buộc phải ra trường sau đúng 4 năm (8 kỳ), dù em đã đủ điều kiện ra trường sau kỳ 1 năm 4. Chính vì thế em quyết định không làm/nộp KLTN mà học 3 môn thay thế. Nếu em có thể đăng ký KLTN cùng lúc với DA2 (có thể hơi khó), em sẽ cân nhắc thực hiện ạ.

Về vấn đề quá chú trọng infra trong DA1, em đã hiểu ý cô và em sẽ đẩy bớt về đồ án 2.

Ở đồ án 1, em sẽ chỉ deploy bằng Aspire lên dịch vụ cloud như Azure chứ không can thiệp sâu vào hạ tầng. Em sẽ tập trung vào nghiệp vụ nghe nhạc hơn, cụ thể là:

- Core feature nghe phát, playlist, share, search. Phân theo artist, thể loại,...
- Lyrics (có thể tích hợp lời tính từ đầu trước)
- Hệ thống transcode và stream nhạc. Nếu có thể và băng thông cho phép, stream trực tiếp FLAC để hỗ trợ nhạc lossless. Transcode async trên một service riêng với RabbitMQ tránh nghẽn server chính, em nghĩ như thế là ổn.
- Phủ test. Coverage càng cao càng tốt.
- Hệ recommendation cơ bản, có thể gồm:

- Trending
- Discovery (em chưa có logic cụ thể ngoài random bài hát)
- Radio Bài hát, recommend thêm theo thể loại, artist,...
- Mix generated playlist (history, related, other people's history)

Em chỉ dự tính tạm hệ RS cơ bản sẽ là như thế, tuy nhiên em muốn chuyển sang RS nâng cao hơn, ví dụ sử dụng <https://github.com/recommenders-team/recommenders> hoặc keras-rs. Hơi tiếc phần này em chưa có kinh nghiệm nên có thể sẽ không kịp, phải để lại DA2.

Sau đó, ở đồ án 2, em sẽ đưa deploy qua mô hình on-premise chạy cluster K8s thủ công, và mở rộng thêm:

- Tách thành microservice
- Thêm lyrics time-synced

- Stream CDN
- statistics và report cụ thể hơn
- RS nâng cao hơn như đề cập ở trên
- Listen together (sync playback của nhiều người, queue chung,...) và hệ thống social/comment.

Tuy nhiên, em kẹt ở một nghiệp vụ: upload nhạc.

Nếu em cho phép tất cả mọi người up nhạc (giống Soundcloud) thì nền tảng sẽ rất hỗn hợp tạp nham, và rất nhiều "bài hát" chất lượng thấp.

Nếu em chỉ cho phép "record label" đăng nhạc (Spotify,...) thì sẽ rất khó tiếp cận thị trường indie và artist nhỏ, khó mở rộng thư viện.

Em đang suy nghĩ sẽ "hybrid", cho phép record label đã xác minh (admin verify thủ công) đăng nhạc trực tiếp, nhưng cho phép artist lẻ (lập tài khoản thoải mái, chỉ cần verify danh tính) đăng nhạc với mức độ ưu tiên thấp hơn các bài nhạc official bởi label. Sau khi đạt đủ lượt nghe/follow nhất định có thể apply lấy "dấu tick" để có lượt ưu tiên ngang với label. Cô nghĩ sao về cách xử lý này?

Em cảm ơn cô,

Hồ Nguyên Minh

[Quoted text hidden]

Trần Thị Hồng, Yến <yentth@uit.edu.vn>

Tue, Jan 6, 2026 at 10:43 PM

To: "Nguyễn Minh, Hồ" <23520923@gm.uit.edu.vn>

Hi Minh,

Em tiếp tục thực hiện Đồ án 1, 2 và học 3 môn thay thế theo kế hoạch mà không thực hiện KLTN cũng được.

Giải quyết bài toán Quy trình Upload nhạc (Hybrid Model): em đang đi đúng hướng với mô hình **Hybrid**. Việc cho phép cả Label và Indie Artist hoạt động là xu thế bắt buộc (giống Spotify hiện nay đã mở Spotify for Artists, hay Soundcloud chuyển mình). Tuy nhiên, khái niệm "mức độ ưu tiên thấp" cần được cụ thể hóa bằng logic kỹ thuật và thuật toán. Dưới đây là gợi ý triển khai cụ thể cho mô hình này:

A. Phân loại Account (User Roles)

Em nên thiết kế hệ thống Role rõ ràng:

1. **Listener:** Chỉ nghe, tạo playlist cá nhân.
2. **Unverified Artist (Indie/Newbie):** Đăng ký tự do, cần verify email/SĐT.
3. **Verified Artist:** Đã đạt KPI hoặc được duyệt thủ công (có tích xanh).
4. **Label/Admin:** Quản lý nhiều nghệ sĩ, độ tin cậy tuyệt đối.

B. Cơ chế "Sandbox" (Hộp cát) thay vì "Ưu tiên thấp"

Thay vì chỉ nói là ưu tiên thấp, em hãy áp dụng cơ chế **Sandbox & Visibility Scope**:

- **Đối với Label/Verified Artist:**

- Nhạc upload lên -> Transcode -> **Public ngay lập tức**.
- Index vào hệ thống Search chính.
- Được đề xuất bởi thuật toán Recommendation System (RS) ngay lập tức.
- Thông báo đến người theo dõi (Push Notification).

- **Đối với Unverified Artist (Indie mới):**

- Nhạc upload lên -> Transcode -> **Trạng thái: "Discover Mode" (hoặc Unlisted)**.
- **Giới hạn tiếp cận:** Bài hát vẫn nghe được, có link chia sẻ, nhưng không xuất hiện trên trang chủ, không hiện trên top trending chung, và không được thuật toán RS gợi ý cho user đại trà.
- **Cách thoát Sandbox:** Bài hát phải tự kiếm được lượng traffic tự nhiên (artist tự share link cho fan, bạn bè). Khi đạt ngưỡng (ví dụ: >1000 lượt nghe unique, >50 like), hệ thống tự động trigger job chuyển trạng thái sang **Public** -> lúc này mới được RS quét và đề xuất.

Nếu chọn cách này em có thể:

- 1. **Chống rác hệ thống RS:** Recommendation System của em sẽ không bị nhiễu bởi các bài test, nhạc lỗi, nhạc kém chất lượng của user mới.
- 2. **Động lực cho Artist:** Buộc artist mới phải tự đi promote nhạc của mình ban đầu (giống thực tế).
- 3. **Giải quyết vấn đề "tạp nham":** Người nghe bình thường vào trang chủ sẽ chỉ thấy nhạc chất lượng (từ Label hoặc Indie đã qua kiểm chứng bằng view thật).

C. Vấn đề bản quyền (Copyright) - Simplified cho Đồ án

Em không thể làm hệ thống quét bản quyền như Content ID của YouTube (quá khó cho ĐA1), nhưng em có thể xử lý ở mức UI/UX và Policy:

- Khi Artist upload, bắt buộc tick vào checkbox: *"Tôi cam kết sở hữu bản quyền bài hát này. Nếu vi phạm, tài khoản sẽ bị khóa vĩnh viễn."*
- Thêm chức năng **"Report"** ở trình nghe nhạc. Nếu 1 bài hát của Indie Artist bị report quá N lần -> Tự động ẩn bài hát và gửi cảnh báo cho Admin xem xét.

Lời khuyên cho ĐA1:

- **Streaming FLAC** (Lossless) tốn băng thông rất lớn => Em nên giới hạn bitrate (ví dụ: tối đa 320kbps cho user thường, FLAC cho VIP/Premium giả lập) để demo tính năng phân quyền luôn.
- **Storage Tier:** Khi deploy lên Azure, hãy chú ý chi phí. Em nên set policy cho Azure Blob Storage. Nhạc gốc (FLAC) để ở *Cool Tier* (ít truy cập), nhạc stream (MP3/AAC) để *Hot Tier*.
- **Unit Test:** Với logic Hybrid Upload này, em sẽ có rất nhiều case hay để viết Unit Test (ví dụ: Test xem user thường upload xong thì status có đúng là Sandbox không? Test xem khi view > 1000 thì có bắn event promote không?).
- **Documentation:** Hãy vẽ Flowchart rõ ràng cho quy trình Upload này và đưa vào báo cáo. Đồ án sẽ được đánh giá cao nếu em có quy trình kiểm soát chất lượng nội dung (Quality Control Workflow).
- Cách xử lý **Hybrid + Sandbox** như cô gợi ý ở trên sẽ giúp em cân bằng được giữa việc "làm giàu thư viện nhạc" và "giữ sạch nền tảng". Nó khả thi để code trong ĐA1 mà không cần AI phức tạp.

Cô Yến.

[Quoted text hidden]

Trần Thị Hồng, Yến <yentth@uit.edu.vn>
To: "Nguyễn Minh, Hồ" <23520923@gm.uit.edu.vn>

Sun, Feb 1, 2026 at 6:44 PM

Em làm gấp Đề cương chi tiết Đồ án 1 theo mẫu quy định chung cho ĐA1, 2 và KLTN, rồi gửi Cô xét duyệt nội dung trước khi thực hiện nha.

[Quoted text hidden]



Đề cương chi tiết khóa luận tốt nghiệp.pdf
200K

Nguyễn Minh, Hồ <23520923@gm.uit.edu.vn>
To: "Trần Thị Hồng, Yến" <yentth@uit.edu.vn>

Tue, Feb 3, 2026 at 3:29 AM

Kính chào cô,

Em xin lỗi cô vì nộp đề cương chậm trễ ạ. Mong cô thông cảm

Em có đính kèm trong mail này bản nháp đề cương của em ạ.

Em cảm ơn cô,

Hồ Nguyễn Minh

[Quoted text hidden]



Trần Thị Hồng, Yến <yentth@uit.edu.vn>
To: "Nguyễn Minh, Hồ" <23520923@gm.uit.edu.vn>

Tue, Feb 3, 2026 at 11:44 PM

Chào Minh,
Cô có một số trao đổi như sau:

- **Streaming Audio:** Đề cương chưa nói rõ cơ chế phát nhạc. Sẽ dùng Range Requests (HTTP 206) đơn giản hay HLS/DASH (Adaptive Streaming)? Nếu chỉ upload file mp3 lên rồi tải về nghe thì chưa gọi là "hệ thống phát nhạc" đúng nghĩa.
- **Bảng kế hoạch thực hiện:**
 - **Giai đoạn "Cài đặt xử lý nghiệp vụ" (23/03 – 25/04):** 1 tháng để code toàn bộ backend + frontend logic nghe nhạc + playlist + search. Khá gấp gáp nếu làm một mình.
 - **Giai đoạn "Triển khai" (06/05 – 10/05):** Chỉ có 5 ngày để "Deploy lên production, cấu hình CI/CD". Việc dựng pipeline CI/CD để build Docker image và push vào K8s cluster chạy ổn định thường tốn nhiều thời gian hơn thế.
 - Em xem phân bổ thêm thời gian hợp lý nha.

Cô Yến.

[Quoted text hidden]

Nguyễn Minh, Hồ <23520923@gm.uit.edu.vn>
To: "Trần Thị Hồng, Yến" <yentth@uit.edu.vn>

Wed, Feb 4, 2026 at 1:15 PM

Em chào cô,

Về streaming audio, em đã thêm vào đề cương yêu cầu chức năng stream nhạc qua adaptive streaming.

Về bảng kế hoạch,

- Thứ nhất, em đã sửa timeline chung lại cho phân bổ thêm thời gian vào hai mục thiết kế UI/UX (nhằm giảm thiểu thời gian code frontend thật sự, em không sử dụng AI "vibe code" nên một file Figma bài bản sẽ giúp em code frontend nhanh hơn)
- Thứ hai, em đã có kinh nghiệm triển khai môi trường K8s-like (ACA, Terraform) ở môn học cloud kỳ trước nên em đã có sẵn nền/template cần thiết để deploy. Em nghĩ 5 ngày để adapt sang đề án mới này là có thể ạ. Đồng thời, em đã có tên miền riêng và SSL certificate wildcard nên phân đoạn deploy sẽ không cần chờ cấp. Chính vì thế em nghĩ 5 ngày để deploy là đủ ạ.

Em xin đính kèm gửi cô file đã cập nhật ạ. Em cảm ơn cô,

Hồ Nguyễn Minh.

[Quoted text hidden]



Trần Thị Hồng, Yến <yentth@uit.edu.vn>
To: "Nguyễn Minh, Hồ" <23520923@gm.uit.edu.vn>

Wed, Feb 4, 2026 at 4:04 PM

Đề cương OK rồi. Em triển khai thực hiện luôn nhé!
Cô Yến.

[Quoted text hidden]

Nguyễn Minh, Hồ <23520923@gm.uit.edu.vn>
To: "Trần Thị Hồng, Yến" <yentth@uit.edu.vn>

Tue, Mar 10, 2026 at 9:53 PM

Em chào cô,

Trước hết, em quên mất về việc báo cáo 2 tuần một lần trong tuần lễ tết do em tưởng tuần nghỉ không tính vào báo cáo, em có hỏi bạn và hiện giờ em mới gửi mail đầu tiên ạ, em xin lỗi cô.

Đây sẽ là mail đầu tiên báo cáo tiến độ của em, mong cô đọc và cho em ý kiến ạ.

Về tiến độ hiện tại, trong hai tuần vừa rồi sau tết, em đã tìm hiểu nền công nghệ cần thiết cho đồ án và các sản phẩm liên quan. Trong đó:

Thứ nhất, về nền công nghệ, do nền chính là web, em đang tìm hiểu các thư viện adaptive streaming phù hợp. Hiện industry-standard có hai protocol chính là HLS và DASH, tuy nhiên thông tin cụ thể về hai protocol và tài liệu so sánh vẫn còn thiếu hụt, không cụ thể (chung chung cho "dân kinh tế") hoặc outdated. Theo em rút ra được, HLS là công nghệ lâu đời hơn, và được hỗ trợ rộng rãi hơn nhưng nền tảng chính vẫn là Apple, còn DASH thì lại là chuẩn chung hơn, không quan tâm codec streaming (codec-agnostic) nên theo em sẽ dễ hỗ trợ stream các format mới hơn (FLAC hoặc đồng thời cả OPUS, OggVorbis,...), tuy nhiên lại không được hỗ trợ bởi các thiết bị của Apple (chỉ áp dụng khi em muốn mở rộng ra app mobile native sau này). Em vẫn phân vân giữa hai thư viện, cả hai đều có thư viện client web tương ứng (hls.js và dash.js) tuy nhiên hls.js có vẻ được sử dụng nhiều hơn, 4 triệu download hàng tuần so với 500k của dash.js trên NPM. Em sẽ xong sớm MVP thử nghiệm stream qua hai protocol trong xấp xỉ 2 ngày tới.

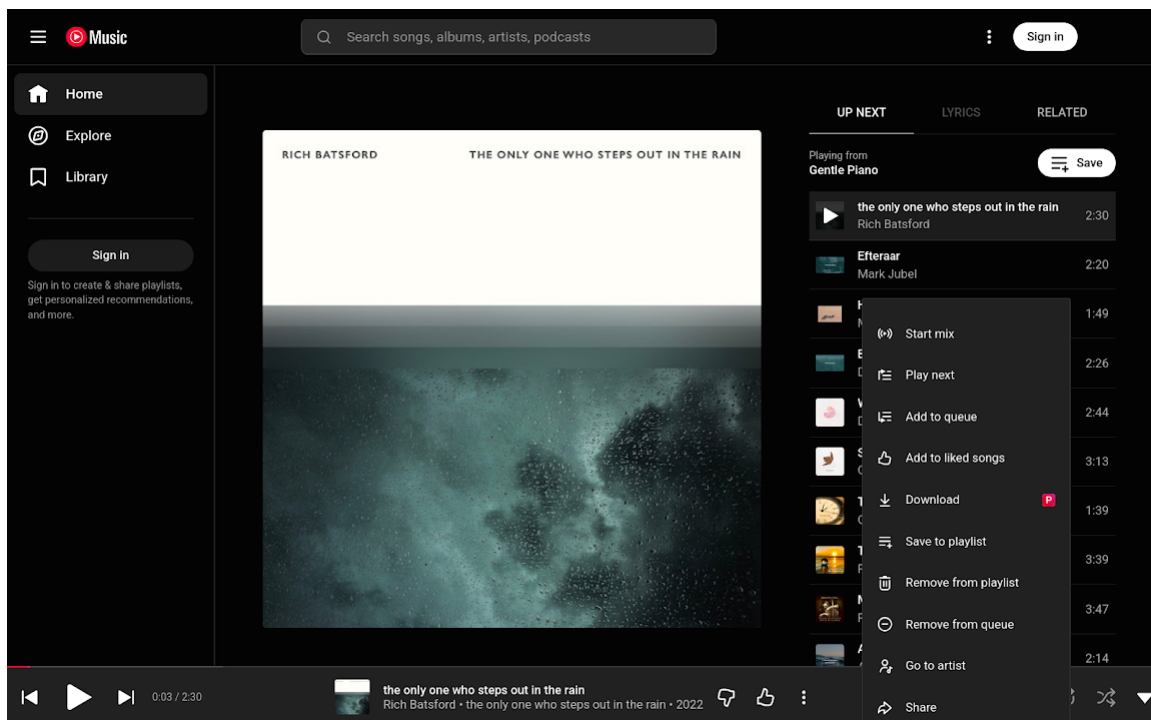
Đồng thời, em cũng đang tìm hiểu và xây dựng hạ tầng cho app. Em đã tạo thành công tài khoản Oracle Cloud, cho phép hạ tầng 4OCPU và 24GB RAM, và đang triển khai một cluster K3s ở trên phần cứng đó. Em có đính kèm hình output kiểm tra health của cluster ở dưới (tạm thời chỉ một node, em sẽ thêm node vào ở đoạn cuối dự án để tăng khả năng chịu tải). Do đang kẹt phần Ingress và tích hợp Load balancer của Oracle vào nền vẫn chưa hoàn thiện hết, dự tính hạ tầng cơ bản sẽ xong vào ngày mai (11/3).

```
someone@federal-device:~$ ssh opc@skynet-alpha.honguyenminh.io.vn
Last login: Tue Mar 10 09:30:03 2026 from 103.199.40.144
[opc@skynet-alpha ~]$ sudo k3s kubectl get nodes
NAME                STATUS    ROLES                  AGE     VERSION
skynet-alpha        Ready     control-plane          4h5m    v1.34.5+k3s1
[opc@skynet-alpha ~]$
```

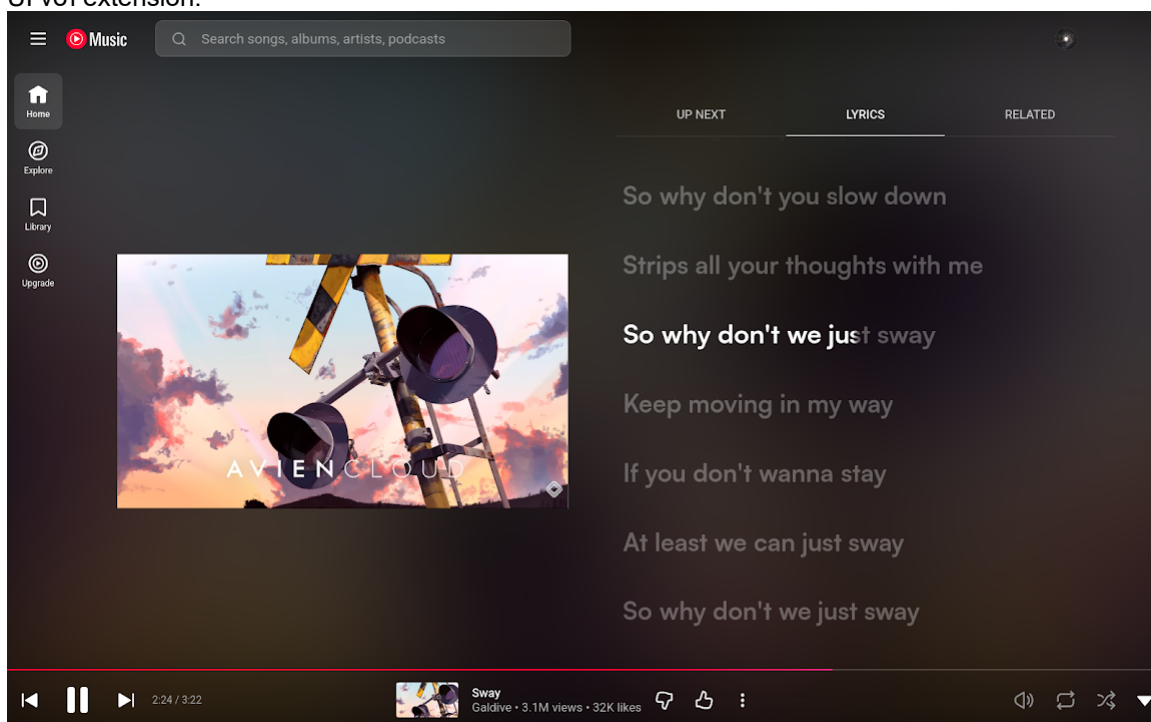
Thứ hai, về các sản phẩm liên quan, và tính năng cụ thể của đồ án.

Hai cái tên chính mà em muốn học hỏi là Youtube Music (streaming side) và Bandcamp (selling side). Em không thích Spotify vì lí do cá nhân và vấn đề với mô hình hoạt động công ty nên sẽ bỏ qua. Về YT Music (em sẽ viết tắt là YTM), các tính năng UI chính và recommend họ làm rất tốt sẵn rồi, tuy nhiên kết hợp thêm extension Better Lyrics, UI của họ thật sự đẹp, theo hướng Minimalism/Clean.

UI mặc định:



UI với extension:



Tuy nhiên, player mặc định của họ thiếu một tính năng khá phiền chính là **normalize âm lượng**, giúp cố gắng chỉnh âm lượng cảm nhận được của các bài hát thành na ná nhau để tránh "giật mình do âm lượng đổi đột ngột khi đổi bài". **Em sẽ cố thêm tính năng này.**

Ngoài ra, một tính năng em nghĩ ra, lấy cảm hứng từ quy trình nghe nhạc cổ điển, **chọn lọc list nhạc cho session nghe**, tính năng sẽ mở rộng thêm tính năng hàng đợi (Queue) truyền thống. Sẽ có một trang UI chuyên cho việc pick các bài nhạc sẽ nghe trong phiên nghe nhạc này, với phần trung tâm là các Album/Mixtape/EP từ thư viện nhạc (người dùng thêm yêu thích, thêm playlist, mua, wishlist), và một xác suất nhỏ hơn các bài hát "mới lạ" được recommend (tuy nhiên khả năng cao hệ khuyến nghị sẽ không kịp trong đồ án này, em sẽ nói thêm ở bên dưới). Người dùng sẽ có 5-10 giây "nghe thử", và có thể thêm vào danh sách chờ hiện tại, hoặc không thêm. Tính năng này giúp người dùng chủ động chọn nội dung muốn nghe trong phiên hiện tại mà không cần phụ thuộc vào playlist (cần tự maintain, và mang tính cố định khó thay đổi theo mood), hoặc một hệ thống random hoàn toàn như các dịch vụ hiện có trên thị trường (không phản ánh đúng nhạc muốn nghe ngay lúc đó).

Về UI cụ thể, có thể là dạng 3D mô phỏng lại các "đĩa nhạc" xếp chồng nhau flip dần ra để mình nghe thử, quét phải để pick và trái để discard qua chồng "bỏ". Cá nhân em khá thích cách này nhưng tính khả thi cho việc thực hiện sẽ hơi thấp do cá nhân em chuyên về backend và hạ tầng hơn là frontend (em không đủ trình độ và tự tin để

code đúng chính xác concept như thế). Mockup em vẽ "tạm" cho một UI như thế. UI này phù hợp hơn cho các thiết bị cảm ứng do các gesture như quét lên để xem bài, quét phải để chọn bài,... tuy nhiên thay thế bằng nút trên PC hoàn toàn được. Các đĩa nhạc kế tiếp sẽ có hiệu ứng 3D như "lật trang sách".



Hoặc là một giao diện hai list cuộn dọc, list trái để chọn nhạc (vẫn design đĩa nhạc để bắt mắt hơn), list phải là đã chọn. Cách làm này sẽ dễ implement hơn nhiều so với UI phía trên.

Thứ hai, hiện tại em phân vân có nên làm một design minimalist giống YTM không, hay phá cách và design một web với phong cách lạ mắt hơn. Cảm hứng chính của em là các loại design theo phong cách Cyberpunk, Retro, Old school (phong trào indie web gần đây, UI và poster design của game Zenless Zone Zero,... chủ yếu nổi bật bởi công nghệ cũ như TV CRT, Cassette, CD, comics book, kết hợp với nét công nghệ mới, phong cách BOLD và design phá cách Dadaism) hoặc Classical Victorian, như UI của game Reverse:1999 hoặc trang web quảng cáo của game LoL Star Guardian (starguardian.com, hiện web đã sập nhưng có thể xem lại video quay tại <https://youtu.be/HDy3MvwsUsl>). Nếu design mới, có thể sẽ tốn thời gian rất nhiều để làm frontend, và khả năng trẻ hạn khá cao, nhưng sản phẩm sẽ ấn tượng hơn. Nếu theo hướng minimalist design sẽ an toàn hơn (cá nhân em có kinh nghiệm design theo Material Design 3 của Google), nhưng sản phẩm cuối tối đa sẽ giống với các sản phẩm hiện tại.

Thứ ba, do bản chất app là bán (có role người bán) nên cần hệ thống thanh toán có khả năng payout (chi hộ, lấy tiền từ tài khoản chung của app để trả tiền công tự động) sau hoặc mỗi chu kỳ tính phí cho stream, hoặc 1-2 ngày ngay sau khi có giao dịch mua. Tuy nhiên, các cổng thanh toán hiện tại trên thị trường Việt Nam không có lựa chọn cổng thanh toán có chi hộ cho các tenant nhỏ lẻ như sinh viên, hộ nhỏ,... mà chỉ hỗ trợ doanh nghiệp có đăng ký giấy phép kinh doanh và MST, em không thể đăng ký.

Trường hợp duy nhất là PayOS, lại:

1. hỗ trợ chi hộ phí rất cao và rất bất ổn định, đồng thời hoạt động dựa trên nền là ví Bảo Kim, ví điện tử mà có trải nghiệm em xin phép nói là tệ nhất từ trước tới nay em tương tác với các nền tảng finance online, và:
2. yêu cầu thẻ MBBank, em hiện không có thẻ ở ngân hàng này, và không có lựa chọn sandbox để "thử nghiệm dev".

Em xin phép propose một trong hai alternative, hoặc là em sử dụng một "mock payment gateway" giả em đã viết từ trước, emulate lại các API thường được cung cấp bởi các payment gateway, hoặc em dùng PayPal sandbox nhưng chỉ hỗ trợ VISA/Mastercard/Paypal.

Cuối cùng, về hệ khuyến nghị, do thường hệ khuyến nghị yêu cầu runtime đặc biệt khi deploy (GPU, phần cứng khỏe, stack Hadoop/Spark/Kafka/Airflow,...) và yêu cầu dữ liệu để train ban đầu, mà đây chưa phải chuyên môn em đã rõ từ trước, nên khả năng cao em sẽ không làm sâu và làm kịp trong đồ án này được. Em hiện không rõ hướng đi cho tính năng này ạ, mong cô góp ý.

Em cảm ơn cô,
Hồ Nguyên Minh.

[Quoted text hidden]

Chào Minh,

Chào em, cô đã xem báo cáo tiến độ 2 tuần đầu Đồ án 1 của em. Đề tài "Xây dựng hệ thống phát nhạc trực tuyến trên nền tảng Cloud Native sử dụng .NET và Kubernetes" là một đề tài khó, đòi hỏi kiến thức hạ tầng và backend vững. Báo cáo em viết có chiều sâu, thể hiện tư duy nghiên cứu công nghệ bài bản và kỹ năng nhận diện rủi ro tốt. Dưới đây là nhận xét chi tiết và định hướng của Cô để giúp em tháo gỡ các vướng mắc hiện tại:

1. Về Giao thức Streaming (HLS vs DASH): Việc em tự tìm hiểu, so sánh chi tiết ưu/nhược điểm của HLS (tính phổ biến, hệ sinh thái Apple) và DASH (chuẩn mở, hỗ trợ nhiều codec như FLAC, OPUS) cho thấy em thực sự hiểu bản chất bài toán streaming. Kế hoạch viết MVP để test thử cả hai giao thức trong 2 ngày tới là một hướng đi thực tế tốt. Vì em làm hệ thống phát nhạc, nếu định hướng hỗ trợ nhạc chất lượng cao (Lossless/Hi-Res), DASH sẽ nhỉnh hơn. Tuy nhiên, nếu ưu tiên độ ổn định trên trình duyệt và thời gian triển khai nhanh, hls.js là lựa chọn an toàn. Em cứ hoàn thành MVP rồi đưa ra quyết định cuối cùng.

2. Về Cổng thanh toán và Tính năng Payout: Em đã phân tích chính xác khó khăn khi tích hợp cổng thanh toán thực tế ở Việt Nam đối với cá nhân/sinh viên (yêu cầu pháp nhân, phí cao, API thiếu ổn định). Trong phạm vi Đồ án 1, mục tiêu chính là chứng minh luồng xử lý logic của hệ thống thay vì vận hành thương mại thật. Cô đồng ý với đề xuất sử dụng PayPal Sandbox hoặc Tự viết Mock Payment Gateway của em. Cả hai cách này đều minh chứng được là hệ thống của em có khả năng xử lý giao dịch, webhook trả về, và tính toán chia tiền cho người bán/creator mà không bị vướng rào cản pháp lý. Em có thể chọn 1 trong 2 giải pháp này để tiết kiệm thời gian.

3. Về Hệ thống khuyến nghị: Việc em nhận ra các giới hạn về phần cứng (GPU), thời gian, và hạ tầng (Hadoop/Spark/Airflow) để train mô hình AI phức tạp cho thấy em biết cách đánh giá và giới hạn phạm vi dự án. Em không nên bỏ tính năng này, mà hãy giảm độ phức tạp của nó xuống. Thay vì dùng Deep Learning hay Big Data, em có thể áp dụng các phương pháp nhẹ nhàng hơn:

- **Content-Based Filtering (Lọc theo nội dung):** Đơn giản là gợi ý bài hát cùng thể loại (Genre), cùng ca sĩ, hoặc cùng BPM. Em có thể làm điều này bằng các câu truy vấn Database thông thường.
- **Sử dụng ML.NET:** Vì stack Backend của em là .NET, em có thể tích hợp thư viện ML.NET của Microsoft. Nó có sẵn thuật toán Matrix Factorization (Collaborative Filtering) để gợi ý bài hát dựa trên lịch sử nghe của user. Việc train mô hình này trên lượng data nhỏ gọn có thể chạy trực tiếp trên CPU bằng C# mà không cần hệ thống Big Data.

4. Về việc em phân vân có nên làm một design minimalist giống YTM không, hay phá cách và design một web với phong cách lạ mắt hơn, Cô khuyên em nên chọn sự an toàn cho kiến trúc cốt lõi, và dành sự phá cách cho Landing page hoặc các chi tiết nhỏ (Animation, Icon) để tránh rủi ro trễ hạn, làm ảnh hưởng đến công sức em đã bỏ ra cho phần khó Cloud Native ở Backend.

Tiến độ 2 tuần đầu của em như vậy là tốt. Em đã đi đúng hướng khi xác định sớm được các điểm nghẽn để tìm cách giải quyết. Em hãy tiếp tục hoàn thiện MVP Streaming và chọn phương án Mock Payment để tiếp tục thực hiện Đồ án theo kế hoạch và kịp tiến độ nhé!

Cô Yến.

[Quoted text hidden]

Em chào cô,

Về thư báo cáo tiến độ tuần vừa rồi của em hơi chậm trễ do yếu tố ngoài, hiện em gửi mail để tóm tắt tiến độ ạ.

Trong quá trình làm MVP (em quyết định dùng DASH) và thiết kế requirement, em đã nghiên cứu qua về nghiệp vụ thực tế cụ thể của ngành công nghiệp nhạc, và phía bên dưới em xin phép trình bày kết quả nghiên cứu được, và khúc mắc gặp phải:

Thứ nhất, ngành nhạc hiện tại được chia làm 5 actor chính khác nhau, người nghe là consumer chính, các label/group quản lý và đại diện nhạc sĩ, các indie artist không nằm trong một label nào, distributor chuyên upload nhạc

lên các nền tảng stream lớn, và các nền tảng stream làm trung gian giữa các party còn lại.

Chính vì thế, nền tảng stream nhạc bản chất nó là B2B2C, trung gian giữa business (label/group, distributor, indie artist coi như một business một thành viên để đơn giản hóa mô hình) và consumer (người nghe). Chính vì thế, do mức độ phức tạp của nghiệp vụ, việc kết hợp cả hai bên B2B và B2C vào cùng một codebase frontend sẽ khá nặng nề và khó mở rộng. Em đề xuất tách hai cổng ra thành hai frontend khác nhau để dễ quản lý cũng như sử dụng cho user (quản lý thì vào business.music.com, nghe nhạc thì vào music.com chẳng hạn). Một business nói trên sẽ có một Organization, với ít nhất một thành viên có quyền Admin, còn lại sẽ được thêm và cấp quyền cần thiết (kế toán, up nhạc,...) bởi admin.

Tuy nhiên, do một tài khoản đăng nhập (Account trong DB) có thể sử dụng để đăng nhập và truy cập vào nhiều role khác nhau (Listener, quản lý hoặc kế toán cho một hoặc nhiều Organization,...) nên không thể sử dụng hệ thống role như thông thường. Em đề xuất sử dụng Claims based authentication, trong đó refresh token chỉ lưu sub (user id trong bảng account), còn để truy cập vào một trang cụ thể sẽ phải xin access token cụ thể chứa organization name và claims của actor cho organization đó. Để invalidate access token có thể dùng một server redis để lưu token bị invalidate, backend chỉ cần tin tưởng vào claims trong token là được. Mô hình này tuy hơi phức tạp về auth nhưng dễ mở rộng do một account đăng nhập có thể có nhiều "role" khác nhau và vào được nhiều org khác nhau, cũng như có thể dễ thêm OAuth2 để đăng nhập bằng tài khoản social sau này do bảng Account chỉ chịu trách nhiệm quản lý đăng nhập và thông tin cơ bản chứ không lưu thông tin quyền hay thành viên org (sẽ lưu trong bảng khác).

Hệ thống này có thể sẽ hơi tốn thời gian để implement, em đang cân nhắc không biết có nên thực hiện không, do theo đúng nghiệp vụ chuẩn là phải đầy đủ phân quyền như thế này.

Thứ hai, về mô hình cho catalog của web. Cụ thể thì ngành nhạc rất phức tạp trong việc phân chia, do khác với suy nghĩ thông thường, không phải chỉ cần chia ra artist>album>songs là xong:

Đầu tiên là khái niệm release, là một đơn vị phát hành nhạc ra thị trường, gắn liền với một mốc thời gian cụ thể và một "lần" phát hành. Đây có thể là một album, một EP, hoặc một single (SP - single play). Sự khác nhau của ba phân loại này thông thường chủ yếu là thời lượng tổng của release, mức độ nhất quán trong theme,... nhưng quyết định bởi artist hoặc label và không có mốc quy định cụ thể, nên sẽ không có business rule cụ thể cho việc phân loại.

Tuy nhiên, do release là một lần phát hành nhạc tại một mốc thời gian cụ thể, mà nhiều release có thể được release lại sau này (có thể do công nghệ cải thiện nên release lại bản chất lượng cao hơn, remaster, edit, hoặc trong format khác,...) nên sẽ có một release mới ở mốc thời gian khác có chung tên và tracklist, tuy nhiên lại là hai release khác nhau với content thực bên trong (ở trường hợp của chúng ta là file nhạc) khác nhau. Chính vì thế, để group các release có liên quan với nhau lại để dễ tìm, ngành chia ra thêm khái niệm release group. Ví dụ, theo cơ sở dữ liệu MusicBrainz, release group "無罪モラトリウム" đã phát hành album này nhiều lần (1992, 1999, 2008,...) với nội dung khác nhau nhưng tracklist giữ nguyên.

Log in

Create account

Search

Artist

Q

About usProductsSearchDocumentationEnglish

無罪モラトリウム

— Release group by 椎名林檎

OverviewAliasesTagsReviewsDetailsEdit

Album

Release	Artist	Format	Tracks	Country/Date	Label	Catalog#	Barcode
Official							
無罪モラトリウム	椎名林檎	CD	11	JP 1999-02-24	EASTWORLD (Japanese imprint)	TOCT-24065	4988006158429
無罪モラトリウム	椎名林檎	12" Vinyl	11	JP 2008-11-28	EASTWORLD (Japanese imprint)	TOJT-24065	[none]
無罪モラトリウム	椎名林檎	Digital Media	11	XW -	EMI Records (a J-Pop division of Universal Music Japan, 2014–present. Do not use for American/European music!)		602567661191
Pseudo-Release							
Muzai Moratorium	椎名林檎	(unknown)	11	JP 1999-02-24			
Innocence Moratorium	椎名林檎	(unknown)	11	JP 1999-02-24			

Relationships

associated singles/EPs: ここですして。 幸福論 歌舞伎町の女王

Discogs: <https://www.discogs.com/master/563227> [info]

Wikidata: [Q648893](https://www.wikidata.org/wiki/Q648893) [info]

Play on ListenBrainz

Release group information

Artist: 椎名林檎

Type: Album

Rating: (see all ratings)

Tags

Genres: alternative r&b, j-pop, jazz rock, rock

Other tags (none)

See all tags

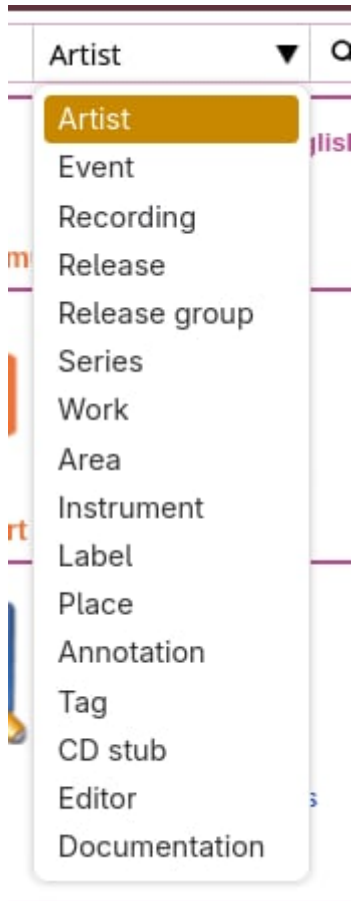
External links

Discogs

Em có nên model mối quan hệ này không? Hay chỉ giữ khái niệm Release đơn giản tách biệt nhau? Do trang cũng bán nhạc nên em muốn có cách để liên kết các album này lại, nhưng nếu model quan hệ one to many cho hai entity này thì rất phiền, do lúc tạo release phải tạo một release group trước,... Hoặc có thể hỗ trợ nhưng giấu đi, cho khoá ngoại trong release đến group là nullable để lúc cần thì có thể tạo chẳng hạn. Hoặc UI tối giản hóa logic, tự tạo một group cho mỗi release, trừ khi tick vào ô release thuộc group đã có sẵn. Phần này em chưa rõ nhưng đang hơi hướng vào phía xử lý UI và làm model đang hoang hơn.

Khái niệm tiếp theo là song. Song là một trường hợp rất khó model, do song là khái niệm trừu tượng, một song có thể có nhiều recording khác nhau rải rác trong nhiều release khác nhau. Ví dụ cũng trường hợp album ở trên, bài "Marunouchi sadistic" có nhiều bản ghi khác nhau, một lần cho album 1992 prerelease, một lần cho album 1999, và một lần rerecord ở album 2008. Đồng thời bên cạnh đó còn có nhiều bản live performance,... Dù cùng một "bài hát" nhưng có nhiều recording khác nhau. Và chúng ta hiện chỉ mới nói nhiều recording khác nhau của cùng một artist, khi vào phạm trù remix, rearrange và cover thì sẽ phức tạp hơn nữa. Em nên model entity này cụ thể thành recording -> song và giữ các remix/rearrange,... thành một song riêng hay cố quan hệ nó lại ạ?

Vẫn còn nhiều khái niệm phức tạp khác, ví dụ danh sách entity chính của music brainz:



Tuy nhiên nếu sử dụng schema của Music Brainz (open source) thì lại quá phức tạp cho nghiệp vụ chính và sẽ kéo dài thời gian rất nhiều nên em chỉ gói lại thành các entity chính phía trên và tạm thời bỏ qua các entity còn lại như series, work, event,...

Còn về streaming, cụ thể em sẽ sử dụng FFmpeg (có thể là một binary server riêng để dễ tách ra và scale cao hơn api thường) để lấy vào file nhạc master được up lên, render thành các stream khác nhau, sau đó encode thành fMP4 và up lên storage (có thể cả CDN nếu cần). Sau đó sinh file mpd cho DASH (và .m3u8 cho HLS nếu cần hỗ trợ sau này) và up lên Storage luôn. Khi có request một bài hát, em sẽ đưa token phát bài hát đó (kỹ giống JWT) để lấy từ storage (Azure Storage SAS) hoặc CDN (Cloudflare CDN Token). Nếu cần host on-prem, có thể tự xây dựng một hệ thống storage có hỗ trợ signing nhưng do đồ án tập trung Cloud-native nên em tạm thời không sử dụng. Em vẫn đang phân vân giữa host bằng cluster K3s ở nhà (phí = không, em toàn quyền kiểm soát), cluster AKS (tích hợp dịch vụ Azure tốt nhưng em chỉ có 100\$ sinh viên đã sử dụng một phần), và cluster OKE (Oracle miễn phí compute hoàn toàn nhưng khó tích hợp với các dịch vụ Storage của azure hơn, có thể phải sử dụng của Oracle cloud, một cloud ít được sử dụng hơn).

Mong được cô xem qua và cho nhận xét,

Hồ Nguyên Minh

[Quoted text hidden]

Chào Minh,
Cô có một số nhận xét và định hướng cho đề tài của em như sau:

1. Kiến trúc Frontend và Xác thực:

- Việc tách Frontend (B2B và B2C): Đề xuất tách [business.music.com](#) và [music.com](#) của em là chính xác về mặt thiết kế hệ thống. Quản lý luồng upload, phân tích thống kê (cho Artist/Label) có UI/UX hoàn toàn khác với luồng nghe nhạc (cho Consumer).
- Claims-based Authentication: Mô hình dùng JWT chứa sub (User ID), kết hợp với Access Token cấp theo từng Organization và Redis Blacklist để invalidate token là tiêu chuẩn công nghiệp hiện nay (OAuth2/OIDC). Framework .NET Core Identity hỗ trợ Claims-based Auth rất mạnh mẽ, nên việc implement ở Backend sẽ không tốn quá nhiều thời gian như em lo lắng.
- Mặc dù Backend nên thiết kế chuẩn Claims ngay từ đầu, ở phía Frontend em chỉ nên làm MVP cho trang Consumer ([music.com](#)) trước. Trang Business có thể làm dạng giao diện admin tối giản hoặc dùng Swagger/Postman để mock các hành động upload nhạc trong giai đoạn này, tránh việc chày timeline vì mãi làm UI.

2. Mô hình hóa dữ liệu Âm nhạc

- Nghiên cứu của em về sơ đồ MusicBrainz rất sâu sắc. Tuy nhiên, nếu em bê nguyên hệ thống MusicBrainz vào Đồ án tập trung vào Cloud Native & Streaming sẽ không kịp tiến độ.
- Hãy đơn giản hóa bằng cách chỉ giữ thực thể Release (với thuộc tính Type: Album, EP, Single). Để giải quyết bài toán các bản remaster/rerelease, em chỉ cần thêm một trường ParentReleaseId (nullable) trỏ về chính bảng Release. Trên UI, nếu người dùng upload bản remaster, họ có thể chọn tick vào bản gốc để gộp nhóm.
- Khái niệm "Song" (tác phẩm trừu tượng) liên quan nhiều đến bản quyền tác giả, trong khi "Recording" (bản thu âm/master file) liên quan đến streaming trực tiếp. Đối với MVP của em, hãy tập trung vào Recording (có thể đặt tên table là Track cho dễ hiểu). Một Track chứa file âm thanh thực tế và nằm trong một Release. Nếu muốn quản lý việc cover/remix, em thêm một trường OriginalTrackId (nullable) vào bảng Track là đủ để giải quyết 80% use-case mà không cần vẽ thêm bảng Song hay Work.

3. Streaming Pipeline và Hạ tầng Cloud

- Quy trình Master File -> FFmpeg -> fMP4 -> DASH (.mpd) -> Storage -> SAS Token của em là một pipeline chuẩn mực của các hệ thống VOD/Audio Streaming hiện đại. Việc tách FFmpeg ra một Worker Node riêng biệt trên Kubernetes (sử dụng Message Queue như RabbitMQ/Kafka để trigger) sẽ phô diễn được tối đa sức mạnh của Cloud Native.
- Lựa chọn Cluster Kubernetes:
 - **AKS (Azure):** Tích hợp rất tốt với Azure Storage, nhưng 100\$ sẽ mất (trong khoảng 1-2 tuần) nếu em chạy Load Balancer và các Node liên tục, em chỉ nên dùng khi chuẩn bị báo cáo đồ án cuối kỳ.
 - **K3s (Local/Home):** Miễn phí, toàn quyền kiểm soát, nhưng em phải tự xử lý việc expose IP ra internet (có thể dùng Cloudflare Tunnels rất tiện lợi và an toàn) và cấu hình persistent volume.
 - **OKE (Oracle):** Oracle có gói Always Free với 4 nhân ARM Ampere A1 và 24GB RAM. Tuy nhiên, kiến trúc ARM64 đòi hỏi các Docker image (nhất là FFmpeg và .NET) phải được build đúng architecture đa nền tảng.
- Định hướng thực tế: Em hãy dùng K3s ở nhà để làm môi trường Development/Testing chính hằng ngày nhằm tiết kiệm chi phí. Khi mọi thứ đã chạy mượt mà, hãy đóng gói toàn bộ Manifests/Helm charts và deploy lên AKS trong 1 tuần cuối cùng trước ngày báo cáo cuối kỳ để demo cho mượt.

Cô Yến.

[Quoted text hidden]